

CSFlow: A Content-Aware Secure Flow Control System for Encrypted Data Sharing in Cloud-Edge

Caiqun Shi , Qinlong Huang , *Member, IEEE*, and Chao Wang 

Abstract—Cloud data sharing facilitates digital collaboration by breaking down information barriers so that all kinds of data can be exchanged between senders and receivers. To resist malicious senders and receivers from sharing data without regulation, flow control systems based on access control encryption (ACE) have been put forward to sanitize encrypted data from the senders at the sanitizers, hence those encrypted data that violates the access control policies will be discarded. Nevertheless, their policies are unrelated to the data content, which may lead to the leakage of sensitive data by authorized senders through encryption. To solve this, we propose CSFlow, a content-aware secure flow control system for fine-grained encrypted data sharing in cloud-edge, which not only controls which senders can communicate with which receivers, but also restricts the content of the communication between them. To instantiate it, we introduce a cross-domain ACE with fine-grained sender policies (CACE), which describes each message with a content tag and assigns each sender a fine-grained sender policy over an attribute vector and a tag within the encryption key. Then, an encrypted data can only be forwarded if its sender policy is satisfied by its receivers and content tag, thus allowing a sanitized ciphertext to be decrypted for any valid receiver by fulfilling the attribute vectors. We give a formal security analysis of selective no-read rule, selective no-write rule, and soundness for CACE, and demonstrate the reasonable efficiency and practicality of CSFlow for cross-domain data flow control in cloud-edge.

Index Terms—Access control encryption, cross-domain, sanitization, fine-grained sender policies, cloud-edge.

I. INTRODUCTION

CLOUD-EDGE computing has been widely adopted for large-scale data sharing, and is especially helpful in the case of cross-domain data sharing, e.g., general hospitals, specialized hospitals, and research institutions share the electronic medical record (EMR) for large-scale prevention of the COVID-19 epidemic. According to privacy protection regulations, e.g., the general data protection regulation (GDPR), it is necessary to protect user privacy and data security in data sharing. Since cloud service providers are semi-trusted and may be curious

Received 8 June 2025; revised 27 April 2026; accepted 14 July 2026. Date of publication 17 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572073 and Grant 62472047 and in part by the Research Program of Beijing University of Posts and Telecommunications under Grant 2025TSQY06. (Corresponding author: Qinlong Huang.)

The authors are with the School of Cyberspace Security and National Engineering Research Center of Disaster Backup and Recovery, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: caiqs@bupt.edu.cn; longsec@bupt.edu.cn; wang_chao@bupt.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3714374>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3714374

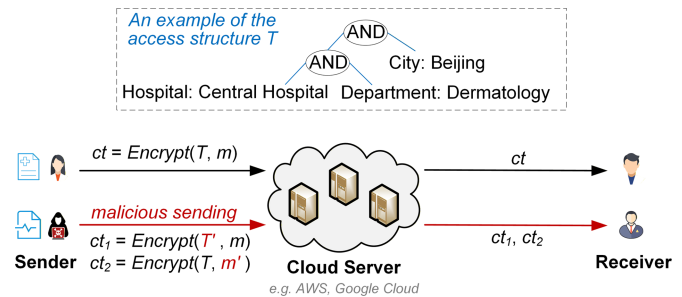


Fig. 1. Medical records sharing in cloud computing.

or profit-driven to leak stored data [1], attribute-based encryption (ABE) [2] technology, including ciphertext-policy ABE (CP-ABE) and key-policy ABE (KP-ABE), especially CP-ABE, has been widely studied for secure data sharing in cloud computing [3], [4]. For example, Waters [5] presented a CP-ABE scheme utilizing linear secret sharing schemes (LSSS) in the random oracle model, Rouselakis et al. [6] put forward two efficient large universe constructions, namely RW-CP-ABE and RW-KP-ABE, and Agrawal et al. [7] presented a fast CP-ABE using the monotone span program (MSP) named FAME. In CP-ABE, the sender encrypts the data using an access structure and uploads the ciphertext to the cloud, as shown in Fig. 1. Only when the receivers' attributes align with the access structure embedded in the ciphertext can they decrypt the ciphertext and gain access to the data. However, malicious senders or compromised devices (e.g., virus-infected computers) may send data to unauthorized receivers, such as using the access structure ("Hospital: Central Hospital AND (Department: Dermatology OR Department: Stomatology) AND City: Beijing") to encrypt the data to enable the users who have the attribute "Department: Stomatology" to access the data. Although attribute-based sign-cryption [8] and matchmaking encryption (ME) [9], [10], [11] technologies enforce the sender authentication, they still cannot stop the sender's malicious behavior, e.g., encrypting data to colluding receivers.

To enforce secure flow control between the sender and receiver, Damgard et al. [12] proposed the concept of access control encryption (ACE), a cryptographic primitive designed to ensure secure data flow control, stipulating which users are allowed to read and write data. Specifically, an access control policy $\mathcal{P}$ is used to define the flow demands of senders and receivers in ACE, which is also utilized by sanitizers to sanitize ciphertexts without decrypting them. The sanitizer checks whether each original

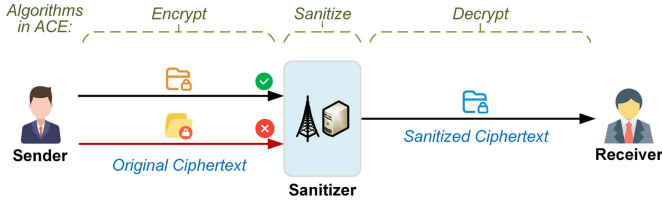


Fig. 2. ACE for secure data flow control.

TABLE I
FINE-GRAINED SENDER POLICIES FOR MEDICAL RECORDS

Sender	Content	Receiver
Alice (Pulmonologist)	COVID-19	Researcher and (NIH or HHMI)
Bob (Neurologist)	Alzheimer's Disease	Professor and (Stanford Univ. or Yale Univ.)

ciphertext complies with the policy and drops it if it does not, as in Fig. 2. For example, only attending physicians at hospitals are authorized to share EMRs with researchers at institutes, while residents are not permitted to do so. Therefore, ACE not only protects against unauthorized decryption by malicious receivers but also prevents malicious senders from transmitting data to unauthorized receivers.

In addition to restricting which senders and receivers can communicate, it is also crucial to impose further restrictions on the content of the data that a sender is permitted to transmit. Content-aware access control schemes [13], [14] have been studied to not only restrict which users can access data, but also restrict what data users can access. Specifically, two-layer authorization is used in [13], the first layer of authorization grants users basic permissions through attributes, while the second layer grants permissions to data based on the content of that data. Unfortunately, these schemes cannot prevent malicious data senders. Currently, many identity-based ACE schemes [15], [16] and attribute-based ACE schemes [17], [18] are proposed to support cross-domain data flow control against malicious senders. However, their access control policies are unrelated to the data content, which still enables authorized senders to inadvertently transmit unauthorized (sensitive) data to legitimate receivers. As a result, there is a critical need to implement fine-grained sender policies for sensitive data that consider not only the receiver's identity or attributes but also the data content, including its classification or sensitivity level. For example, a pulmonologist Alice, is only authorized to send diagnostic records labeled "COVID-19" to relevant researchers at the NIH or HHMI, as shown in Table I. This type of access control policy was first mentioned by Kim et al. [15], but has not been carefully constructed and evaluated yet.

A. Motivations and Challenges

We are interested in content-aware secure flow control in cloud-edge that can support fine-grained sender policies and cross-domain keyless sanitization. In the state-of-the-art

ABACE [18], a digital signature scheme, a non-interactive zero-knowledge proof (NIZKP) system, and a sanitizable encryption scheme are employed to construct a cross-domain ACE scheme with keyless sanitization. Concretely, the sender authority (SA) employs the signature scheme to sign the access structure as the encryption key, certifying the legitimate receivers with whom the sender can communicate. The sender then uses the encryption key to encrypt the data, in which NIZKP is used to prove that the encryption key has been employed to generate the ciphertext, i.e., the NIZKP system anonymously verifies the signature and the ciphertext. To support fine-grained sender policies, it is tempting to follow this construction idea, describe the content allowed by the sender as a tag such as "COVID-19", and finally allow the SA to also sign it when generating the encryption key.

However, this intuitive design cannot prevent the malicious sender from leaking the data containing the sensitive content, because the sanitizer cannot verify the data content sent by the sender in an encrypted form. In addition, in existing ACE schemes [17], [18], [19], [20], the size of the decryption key is linear with the number n of authorized attributes, and $\mathcal{O}(n)$ time-consuming pairing computations are required to decrypt a ciphertext. In summary, the technical challenge we aim to tackle is *How to devise a cross-domain ACE scheme supporting fine-grained sender policies and keyless sanitization with reasonable efficiency, while still using building blocks like signatures, NIZKP, and sanitizable encryption?*

B. Our Contributions

In this paper, we introduce CSFlow, a content-aware secure flow control system designed for fine-grained data sharing in cloud-edge, which not only controls which senders can communicate with which receivers, but also restricts the content of the communication between them. It enforces secure flow control with access control policies to defend against malicious senders and receivers from sharing data without authorization and employs edge nodes to impose content-aware sanitization on the encrypted data from the senders. To do this, CSFlow 1) describes the encryption key with a fine-grained sender policy over both content tag and attribute vector, while producing the decryption key for a predicate vector; 2) introduces a set of supervisors (e.g., the supervisory authorities in GDPR) to collaboratively generate tag-signed messages that certifies each message's content tag using a signature, and requires the sender to get tag-signed messages before sending them. Hence, the sender of a message in CSFlow needs to attest that she/he is allowed to send this type of message to the intended receivers, while an encrypted message could pass the sanitization only if its receivers and content tag conform to the underlying sender's fine-grained sender policy, and the sanitized ciphertext could be decrypted by satisfying its attribute vector. To be specific, the contributions of this work can be outlined as follows.

- We instantiate CSFlow by introducing a new cryptographic primitive named CACE, a cross-domain ACE with fine-grained sender policies. We also give a detailed construction for CACE, from a sanitizable inner-product encryption

(SIPE) scheme, a threshold signature (TSIG) scheme, an equivalence class signature (EQS) scheme, and an NIZKP system.

- We give formal security definitions of selective no-read rule, selective no-write rule, and soundness for CACE. Specifically, soundness ensures that a sender is permitted to transmit only those messages that adhere to their fine-grained sender policies. We also conduct a thorough security analysis of CSFlow in light of the security of the underlying building blocks.
- We perform theoretical and experimental analyses on CACE and compare it with state-of-the-art ACE schemes. The results demonstrate that CACE has efficient encryption, sanitization, and decryption operations, and simultaneously achieves fine-grained sender policies without incurring heavy computational and storage overhead. Therefore, CSFlow is a practical and promising system for secure cross-domain data flow in cloud-edge.

C. Organization

Sections II and III present the related work and preliminaries, respectively. Subsequently, Section IV presents the system model and threat model of CSFlow. Section V introduces the CACE instantiation. Sections VI and VII prove its security and give its performance analysis, respectively.

II. RELATED WORK

To resist malicious senders, Susilo et al. [24] presented a sanitized ABE (SABE) tailored for secure cloud storage and constructed from Waters' CP-ABE scheme [5]. Specifically, the sanitizer checks whether the ciphertext is encrypted according to the claimed access structure and only re-randomizes (sanitizes) the ciphertext that correctly embeds the access structure. To protect the privacy of the receiver, Zhao et al. [25] proposed a ciphertext-policy sanitizable ABE scheme with privacy protection (SABE-PP). Jiang et al. [26] proposed a sanitizable multi-authority ABE (sMABE) based on the multi-authority ABE [27]. However, the above solutions either ignore sender authentication or require a time-consuming computation process to check the access structure of the ciphertext.

To cryptographically enforce information flow, Damgård et al. [12] put forward the concept of ACE, accompanied by two security properties: no-read and no-write rules. Additionally, they provided a construction leveraging cryptographic obfuscation under a global access policy configured at the system setup. Later, a generic construction for global-policy ACE was introduced by Kim and Wu [15], leveraging a combination of digital signature, predicate encryption, and function encryption schemes. Then Han et al. [21] combined ACE with attributes to implement expressive policies, introduced a fine-grained information flow control (FIFC) scheme, and constructed it from an attribute-based signature (ABS) scheme and Waters' CP-ABE scheme [5]. Cui et al. [19] put forward a novel attribute-based ACE (AACE) for fine-grained data sharing in cloud-edge, and constructed it based on an identity-based signature scheme and FAME [7]. Subsequently, Huang et al. [22] presented an efficient attribute-based ACE scheme named AOACE based on

RW-CP-ABE [6] and employed outsourced encryption and decryption technology to alleviate the computational overhead on the users. Xin et al. [28] proposed Hodor, a robust fine-grained information flow control scheme for secure data sharing in cloud-edge, and gave the concrete construction on the basis of RW-CP-ABE [6].

Kim et al. [15] first proposed the notion of dynamic-policy ACE, enabling access control policy to be flexibly tied into encryption/decryption keys. Then Fuchsbaauer et al. [29] presented a dynamic-policy ACE scheme supporting equality, comparison, and interval membership predicates, leveraging a structure-preserving signature (SPS) scheme for key generation and a NIZKP system for ciphertext verification. To support cross-domain data flow control, Wang et al. [16] introduced a cross-domain ACE (CDACE) scheme, featuring identity set-based sender policies and keyless sanitization. They gave a construction from the AGOT-SPS scheme [30], a sanitizable identity-based broadcast encryption scheme, and a NIZKP system. Leveraging RW-KP-ABE, a receiver-policy attribute-based ACE (RA-ACE) scheme was proposed by Huang et al. [20] to realize secure data flow control. In RA-ACE, receiver decryption keys are described with access structures, while sender encryption keys are generated using the AGHO-SPS scheme [31]. Sedaghat et al. [17] extended CDACE [16] by introducing a cross-domain attribute-based ACE (CD-ABACE) scheme with improved policy expressiveness, leveraging Waters' CP-ABE scheme [5] and the AGOT-SPS scheme [30] to create sender keys based on LSSS access structures. However, their scheme exposes the access structure in the ciphertext. Hence, Wang et al. [18] proposed ABACE, an attribute-based ACE scheme that supports policy hiding. To tackle the problem of encryption key escrow, Shi et al. [23] proposed a cross-domain inner-product ACE (CD-IPACE) scheme and gave a concrete construction from a SIPE scheme [32], a threshold SPS scheme [33], and a NIZKP system. To facilitate our motivation understanding, we summarize the comparative characteristics of existing schemes in Table II.

III. PRELIMINARIES

A. Bilinear Pairing

Assume that $\mathbb{G}$, $\mathbb{H}$, $\mathbb{G}_T$ are three cyclic groups of the prime order p , where g is a generator of $\mathbb{G}$ and h is a generator of $\mathbb{H}$. It is required that a bilinear pairing $e : \mathbb{G} \times \mathbb{H} \rightarrow \mathbb{G}_T$ must satisfy the following three characteristics.

- Bilinearity: $e(g^a, h^b) = e(g, h)^{ab}$ holds for all $g \in \mathbb{G}$, $h \in \mathbb{H}$ and $a, b \in \mathbb{Z}_p$.
- Non-degeneracy: $e(g, h) \neq 1$.
- Computability: It is efficient to compute $e(g, h) \in \mathbb{G}_T$ for any $g \in \mathbb{G}$, $h \in \mathbb{H}$.

If $\mathbb{G} = \mathbb{H}$, e is symmetric; otherwise it is asymmetric [21]. In a Type-III bilinear pairing [7], it is impossible to establish an efficiently computable isomorphism between $\mathbb{G}$ and $\mathbb{H}$.

B. TSIG Scheme

A TSIG scheme [34] empowers n parties to collaboratively hold the ability to issue digital signatures, ensuring that any

TABLE II
COMPARISON OF EXISTING SECURE FLOW CONTROL SCHEMES

Schemes	Access structure	Secure write control	Dynamic policy	Keyless sanitization	Cross domain	Policy assignment	Fine-grained sender policies	Ciphertext soundness
FIFC [21]	LSSS	✓	✗	✗	✗	Sanitizer	✗	✗
AACE [19]	MSP	✓	✗	✓	✗	Sanitizer	✗	✗
AOACE [22]	LSSS	✓	✗	✓	✗	Sanitizer	✗	✗
RA-ACE [20]	Attributes	✓	✓	✓	✗	Receiver	✗	✗
CDACE [16]	Identities	✓	✓	✓	✓	Sender	✗	✗
CD-ABACE [17]	LSSS	✓	✓	✓	✓	Sender	✗	✗
ABACE [18]	LSSS	✓	✓	✓	✓	Sender	✗	✗
CD-IPACE [23]	Attribute vector	✓	✓	✓	✓	Sender	✗	✗
CSFlow	Attribute vector	✓	✓	✓	✓	Sender	✓	✓

group of at least $t + 1$ parties can sign, while groups of t or fewer parties cannot. It comprises the following algorithms.

- $\text{KeyGen}(1^\lambda, n, t) \rightarrow (svk, \{ssk_i, svk_i\}_{i \in [n]})$. This algorithm takes as input a security parameter λ , the number n of signers and a threshold t , then outputs a verification key svk , n partial verification keys $\{svk_i\}_{i \in [n]}$ and n signing key $\{ssk_i\}_{i \in [n]}$. Specifically, the j -th signer holds $(svk, \{svk_i\}_{i \in [n]}, ssk_j)$.
- $\text{PSign}(ssk_i, m) \rightarrow \delta_i$. This algorithm takes as input a signing key ssk_i , a message $m \in \{0, 1\}^\ell$, then outputs a partial signature δ_i .
- $\text{Comb}(S, m, \{(svk_i, \delta_i)\}_{i \in S}) \rightarrow \delta / \perp$. This algorithm takes as input a set S with $|S| \geq t + 1$, a message m , a set of tuples $\{(svk_i, \delta_i)\}_{i \in S}$, and then outputs a signature δ or $\perp$.
- $\text{Verify}(svk, m, \delta) \rightarrow 0/1$. This algorithm takes as input svk , a message m , and a signature δ , then outputs 1 if δ is valid and 0 otherwise.

The correctness of TSIG requires that $1 \leftarrow \text{Verify}(svk, m, \delta)$, where $(svk, \{ssk_i, svk_i\}_{i \in [n]}) \leftarrow \text{KeyGen}(1^\lambda, n, t)$, $\delta \leftarrow \text{Comb}(S, m, \{(svk_i, \delta_i)\}_{i \in S})$, and $\delta_i \leftarrow \text{PSign}(ssk_i, m)$. A TSIG scheme is considered to be existentially unforgeable under chosen-message attacks (EUF-CMA) if no probabilistic polynomial-time (PPT) adversary that compromises at most t signers, can produce a valid pair (m^*, δ^*) such that m^* was never submitted as a query to the partial signing oracle. The partial signing oracle, when queried with a message m , returns $\text{PSign}(ssk_i, m)$. That is, a TSIG scheme remains unforgeable even when up to t signers are compromised by an adversary.

C. EQS Scheme

An EQS scheme [35] signs an arbitrary representative $\mathcal{K}$ of an equivalence class $[\mathcal{K}]$, and allows one to choose different representatives and to update corresponding signatures publicly by introducing a new ChgRep algorithm. Specifically, ChgRep can be interpreted as a consistent randomization of a signature and its associated message via the randomizer ρ , while preserving the validity of the signature for the equivalence class. It comprises the following four algorithms.

- $\text{KeyGen}(1^\lambda, \chi) \rightarrow (esk, evk)$. This algorithm takes as input a security parameter λ and the message length χ , then outputs the signing key esk and verification key evk .

- $\text{Sign}(esk, \mathcal{K}) \rightarrow \mu$. This algorithm takes as input esk , a message $\mathcal{K} \in \mathbb{G}^\chi$, then outputs the signature μ on the equivalence class $[\mathcal{K}]$.
- $\text{ChgRep}(evk, \mathcal{K}, \mu, \rho) \rightarrow \mu'$. This algorithm takes as input $evk, \mathcal{K}, \mu$ and the scalar ρ , then outputs the fresh signature μ' on the equivalence class $[\mathcal{K}^\rho] = [\mathcal{K}]$.
- $\text{Verify}(evk, \mathcal{K}, \mu) \rightarrow 0/1$. This algorithm takes as input $evk, \mathcal{K}$ and μ , outputs 1 if μ is valid, otherwise 0.

The correctness of EQS guarantees that if a signature $\mu \leftarrow \text{Sign}(esk, \mathcal{K})$ is generated for a representative $\mathcal{K}$, we always have $\text{Verify}(evk, \mathcal{K}^\rho, \text{ChgRep}(evk, \mathcal{K}, \mu, \rho)) = 1$. Meanwhile, we will require the EQS scheme to provide EUF-CMA security and perfect adaptation. In detail, an EQS scheme is considered to be EUF-CMA secure if no PPT adversary can produce a valid pair $(\mathcal{K}^*, \mu^*)$ that corresponds to an equivalence class $[\mathcal{K}^*]$ that has not been queried in the signing oracle. The signing oracle, when queried with a message $\mathcal{K}$, returns the signature $\text{Sign}(esk, \mathcal{K})$ on the equivalence class $[\mathcal{K}]$. Moreover, an EQS scheme satisfies perfect adaptation if for all tuples $(evk, \mathcal{K}, \mu, \rho)$ such that $\text{Verify}(evk, \mathcal{K}, \mu) = 1$ and $\rho \leftarrow \mathbb{Z}_p$, it holds that $\text{ChgRep}(evk, \mathcal{K}, \mu, \rho)$ and $\text{Sign}(esk, \mathcal{K}^\rho)$ are identically distributed.

D. SIPE Scheme

In SIPE [23], the message is encrypted by an attribute vector $\vec{v}$, while the secret key is generated with a predicate vector $\vec{u}$. The message can be recovered from the ciphertext only if the inner product of $\vec{v}$ and $\vec{u}$ is equal to zero. Moreover, it provides a Sanitize algorithm to rerandomize the ciphertext. Generally, it includes the following five algorithms.

- $\text{Setup}(1^\lambda) \rightarrow (pk_{\text{SIPE}}, mk_{\text{SIPE}})$. This algorithm takes as input a security parameter λ , then outputs the public parameters pk_{SIPE} and the master secret key mk_{SIPE} .
- $\text{KeyGen}(pk_{\text{SIPE}}, mk_{\text{SIPE}}, \vec{u}) \rightarrow sk_{\vec{u}}$. This algorithm takes as input $pk_{\text{SIPE}}, mk_{\text{SIPE}}$, and a predicate vector $\vec{u}$, then outputs a corresponding secret key $sk_{\vec{u}}$.
- $\text{Encrypt}(pk_{\text{SIPE}}, m, \vec{v}) \rightarrow c$. This algorithm takes as input pk_{SIPE} , a message m , an attribute vector $\vec{v}$, then outputs the ciphertext c .
- $\text{Sanitize}(pk_{\text{SIPE}}, c) \rightarrow c'$. This algorithm takes as input pk_{SIPE}, c , then outputs a sanitized ciphertext c' .

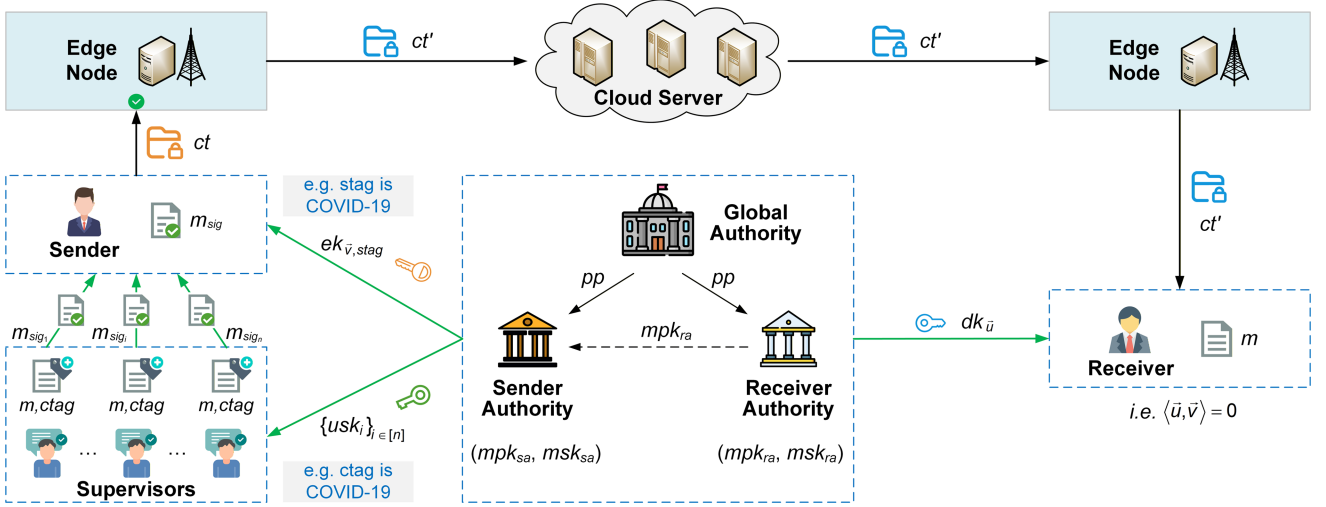


Fig. 3. The system model of CSFlow in cloud-edge.

- $\text{Decrypt}(sk_{\vec{u}}, c') \rightarrow m/\perp$. This algorithm takes as input a secret key $sk_{\vec{u}}$ and a ciphertext c' , then outputs the message m or $\perp$.

The correctness of SIPE requires that for all $(pk_{\text{SIPE}}, mk_{\text{SIPE}})$ generated by $\text{Setup}(1^\lambda)$, all predicate vectors $\vec{u}$, any secret key $sk_{\vec{u}} \leftarrow \text{KeyGen}(pk_{\text{SIPE}}, mk, \vec{u})$, and all attribute vectors $\vec{v}$, if $\langle \vec{u}, \vec{v} \rangle = 0$, then we have $\text{Decrypt}(sk_{\vec{u}}, \text{Sanitize}(pk_{\text{SIPE}}, \text{Encrypt}(pk_{\text{SIPE}}, m, \vec{v}))) = m$. It also should achieve indistinguishability against selective attribute-hiding chosen-plaintext attacks (IND-sAH-CPA) security and weak sanitizability, as defined in [23].

- In particular, we say that an SIPE scheme provides IND-sAH-CPA security if for any PPT adversary who submits the challenge attribute vectors $\vec{v}_0$ and $\vec{v}_1$, challenge messages $\vec{m}_0$ and $\vec{m}_1$, and receives the challenge ciphertext $c_\theta \leftarrow \text{Encrypt}(pk_{\text{SIPE}}, m_\theta, \vec{v}_\theta)$, cannot correctly guess θ with a non-negligible advantage over $1/2$.
- In addition, we say an SIPE scheme provides weak sanitizability if for any PPT adversary who submits the challenge attribute vectors $\vec{v}_0$ and $\vec{v}_1$, messages $\vec{m}_0$ and $\vec{m}_1$, and receives the challenge ciphertext $c'_\theta \leftarrow \text{Sanitize}(pk_{\text{SIPE}}, \text{Encrypt}(pk_{\text{SIPE}}, m_\theta, \vec{v}_\theta))$, cannot correctly guess θ with a non-negligible advantage over $1/2$.

E. NIZKP System

Let $\mathcal{L}$ be a language and $\mathcal{I}$ be a relation which requires for all $\mathcal{V} \in \mathcal{L}$, there always exists a witness $\mathcal{W}$ satisfying $(\mathcal{V}, \mathcal{W}) \in \mathcal{I}$. The following algorithms constitute the NIZKP system [36] associated with relation $\mathcal{I}$:

- $\text{Gen}(1^\lambda, \mathcal{L}) \rightarrow crs$. This algorithm takes as input a security parameter λ and a language $\mathcal{L}$, then outputs a common reference string crs .
- $\text{Prove}(crs, \mathcal{V}, \mathcal{W}) \rightarrow \pi$. This algorithm takes as input the common reference string crs , a statement $\mathcal{V}$, and a witness $\mathcal{W}$, then outputs a proof π .

TABLE III
NOTATIONS

Symbols	Description
pp	The global public parameter
mpk_{ra}/msk_{ra}	The domain public/secret key of RA
mpk_{sa}/msk_{sa}	The domain public/secret key of SA
usk_i	The secret key for the i -th supervisor
$\vec{v}/\vec{u}$	The attribute/predicate vector
$ek_{\vec{v},stag}$	The encryption key for the fine-grained sender policy $(\vec{v}, stag)$
$dk_{\vec{u}}$	The decryption key for $\vec{u}$
m	The message in the system
$ctag$	The content tag of the message
m_{sig_i}	The partial tag-signed message
m_{sig}	The tag-signed message
ct/ct'	The original/sanitized ciphertext

- $\text{Verify}(crs, \mathcal{V}, \pi) \rightarrow \{0, 1\}$. This algorithm takes as input the common reference string crs , a statement $\mathcal{V}$, and a proof π , then outputs 0 or 1.

This system is supposed to be perfectly sound, zero-knowledge, and knowledge-extractable [37]. Specifically, a knowledge extractor can be identified as a pair of PPT algorithms (E_1, E_2) in this system, in which for all PPT adversaries $\mathcal{A}$ we have $\Pr[crs \leftarrow \text{Gen}(1^\lambda, \mathcal{L}) : \mathcal{A}(crs) = 1] \approx \Pr[(crs, \iota) \leftarrow E_1(1^\lambda, \mathcal{L}) : \mathcal{A}(crs) = 1]$, and $\Pr[(crs, \iota) \leftarrow E_1(1^\lambda, \mathcal{L}), (\mathcal{V}, \pi) \leftarrow \mathcal{A}(crs), \mathcal{W} \leftarrow E_2(crs, \iota, \mathcal{V}, \pi) : \text{Verify}(crs, \mathcal{V}, \pi) = 0 \text{ or } (\mathcal{V}, \mathcal{W}) \in \mathcal{I}] \approx 1$.

IV. CSFLOW OVERVIEW

In this section, we delineate the system model, system workflow, and security model. To facilitate understanding, the notations employed in this paper are summarized in Table III.

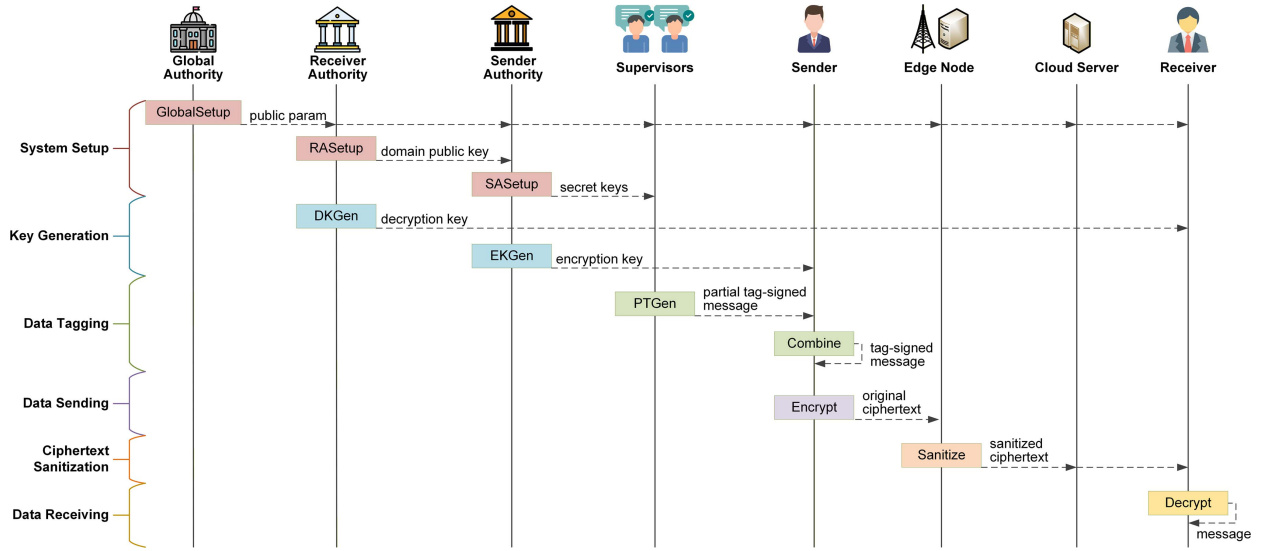


Fig. 4. The workflow of CSFlow.

A. System Model

The architecture of CSFlow is depicted in Fig. 3, involving eight entities: the global authority (GA), SA, receiver authority (RA), the supervisors, the senders, the edge node, the cloud server, and the receivers. CSFlow is built on a standard cloud-edge network architecture, where the edge node monitors and routes all traffic between the senders and the cloud server. The responsibilities of each entity are elaborated as follows.

- **GA:** It generates the global public parameter for the whole flow control system.
- **RA:** It serves as a decryption key generation center, is tasked with generating a unique decryption key for each receiver who registers in the system and overseeing the management of the domain's public and secret keys.
- **SA:** It functions as an encryption key generation center, is responsible for maintaining its own public key and using its secret key to generate a unique encryption key for each sender who registers in the system. It also generates a secret key for each supervisor.
- **Supervisors:** They act as the supervisory authorities in the sending domain consisting of n members. They work together to validate the tag of each data in the system, and issue the appropriate tag-signed data. In Art. 51 GDPR, the supervisory authority is responsible for monitoring the application of this regulation, in order to protect the fundamental rights of natural persons and to facilitate the free flow of personal data within the Union.
- **Sender:** She/he encrypts the tag-signed data with the encryption key, and generates the corresponding original ciphertext, which is then transmitted to the edge node.
- **Edge Node:** It acts as a sanitizer that is responsible for verifying the authenticity of the ciphertext and converting the original ciphertext into a sanitized version. It also alleviates the burden on the cloud server.
- **Cloud Server:** It offers substantial storage capacity for sanitized ciphertexts originating from the edge node.

Additionally, it forwards frequently accessed ciphertexts to the edge node for caching, facilitating faster access.

- **Receiver:** She/he can freely retrieve sanitized ciphertexts of interest from the edge node, and decrypt them with the decryption key to access the protected data.

B. System Workflow

As shown in Fig. 4, the workflow of CSFlow is divided into six phases. The details are as follows.

- **System Setup:** GA first chooses the security parameter to create the global public parameter pp , then RA and SA separately run their setup algorithms to generate the domain public/secret key. Global public parameter and domain public keys are available for all entities, while domain secret keys are only available for the authority. In addition, SA sends the supervisors' secret keys to them through a secure channel.
- **Key Generation:** With the domain secret key, SA/RA runs the encryption/decryption key algorithm to produce the encryption/decryption key for the sender/receiver. Specially, each sender is issued an encryption key for a fine-grained sender policy consisting of an attribute vector and a content tag $stag$.
- **Data Tagging:** For a message, the sender must collect partial tag-signed messages from at least $t + 1$ supervisors. Each supervisor runs the partial tag-signed message generation algorithm to generate a partial tag-signed message m_{sig_i} upon request. Once enough partial tag-signed messages are obtained, the sender runs the combination algorithm to form the final tag-signed message m_{sig} , which embeds the content tag $ctag$.
- **Data Sending:** With the encryption key and tag-signed message, the sender runs the encryption algorithm to produce the original ciphertext ct and sends it to the edge node. Note that the tag-signed message is time-sensitive, so

we require the sender to re-request the tag-signed message when she/he sends the same message later.

- *Ciphertext Sanitization*: After getting the original ciphertext from the sender, the edge node runs the sanitization algorithm to sanitize it. In detail, the edge node verifies the legality of the ciphertext by checking its receivers (described with an attribute vector) and content tag $ctag$ conform to its sender's fine-grained sender policy, especially $stag = ctag$, and converts the original ciphertext into a sanitized version. Finally, the edge node sends the sanitized ciphertext ct' to the cloud server.
- *Data Receiving*: The receiver can request the ciphertext from the nearest edge node, which determines whether the ciphertext exists. If so, the edge node returns it directly to the receiver. Otherwise, the edge node requests the cloud server and returns it to the receiver. After getting the sanitized ciphertext, the receiver invokes the decryption algorithm to recover the message.

C. Threat Model

In CSFlow, senders and receivers are assumed to communicate via public channels, while edge nodes are situated between the senders and the receivers. All original ciphertexts are required to pass through the edge nodes, where they will be transformed into sanitized ciphertexts. In our model, the edge nodes are assumed to be semi-trusted (exhibiting curiosity but no collusion), while both senders and receivers are considered untrustworthy due to possible illicit data sharing.

A malicious sender may try to collude with the unauthorized receiver and leak illegal data, while a malicious receiver may try to collude with the sanitizer or other receivers to access the unauthorized data. Moreover, we think RA and SA are trusted entities and will not collude with other entities, and we assume that the adversary is capable of corrupting up to t supervisors, since some supervisors may not be fully trusted. Additionally, the cloud servers are regarded as semi-trusted, as they may cooperate with other entities to access stored data.

V. CSFLOW INSTANTIATION

CSFlow is instantiated through CACE. We extend the existing definition of ACE to provide a new definition of CACE, adding a partial tag-message generation algorithm and a tag-message combination algorithm to bind a message with a content tag, and inputting the fine-grained sender policy into the encryption key generation algorithm.

A. Formal Definition

A CACE scheme comprises the following ten algorithms.

- $\text{GlobalSetup}(1^\lambda) \rightarrow pp$. This algorithm takes as input a security parameter λ , then outputs the global public parameter pp .
- $\text{RASetup}(pp) \rightarrow (mpk_{ra}, msk_{ra})$. This algorithm takes as input pp , then outputs the domain public key mpk_{ra} and the domain secret key msk_{ra} for the RA.

- $\text{SASetup}(pp, mpk_{ra}, n, t) \rightarrow (mpk_{sa}, msk_{sa}, \{usk_i\}_{i \in [n]})$. This algorithm takes as input pp, mpk_{ra} , the number n of supervisors and a threshold t , then outputs the domain public key mpk_{sa} , the domain secret key msk_{sa} for the SA, and n secret keys $\{usk_i\}_{i \in [n]}$ for the supervisors.
- $\text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}, stag) \rightarrow ek_{\vec{v}, stag}$. This algorithm takes as input mpk_{sa}, msk_{sa} , a fine-grained sender policy consisting of an attribute vector $\vec{v}$ and a content tag $stag$, then outputs an encryption key $ek_{\vec{v}, stag}$ for the sender in the system.
- $\text{DKGen}(mpk_{ra}, msk_{ra}, \vec{u}) \rightarrow dk_{\vec{u}}$. This algorithm takes as input mpk_{ra}, msk_{ra} , and a predicate vector $\vec{u}$, then outputs a decryption key $dk_{\vec{u}}$ for the receiver.
- $\text{PTGen}(mpk_{sa}, usk_i, m, ctag, aux) \rightarrow m_{sig_i}$. This algorithm takes as input mpk_{sa}, usk_i , a message m with its content tag $ctag$, and auxiliary information aux , then outputs a partial tag-signed message m_{sig_i} .
- $\text{Combine}(mpk_{sa}, S, m, \{m_{sig_i}\}_{i \in S}) \rightarrow m_{sig}$. This algorithm takes as input mpk_{sa} , a set S with $|S| \geq t + 1$, a message m and a set of partial tag-signed messages $\{m_{sig_i}\}_{i \in S}$, then outputs a tag-signed message m_{sig} .
- $\text{Encrypt}(mpk_{ra}, mpk_{sa}, ek_{\vec{v}, stag}, m_{sig}) \rightarrow ct$. This algorithm takes as input mpk_{ra}, mpk_{sa} , an encryption key $ek_{\vec{v}, stag}$ and a tag-signed message m_{sig} , then outputs an original ciphertext ct .
- $\text{Sanitize}(mpk_{ra}, mpk_{sa}, ct) \rightarrow ct' / \perp$. This algorithm takes as input mpk_{ra}, mpk_{sa} , an original ciphertext ct , then outputs a sanitized ciphertext ct' if ct is legal, and $\perp$ otherwise.
- $\text{Decrypt}(dk_{\vec{u}}, ct') \rightarrow m$. This algorithm takes as input a decryption key $dk_{\vec{u}}$ and a sanitized ciphertext ct' , then outputs the message m .

The correctness of CACE requires that for any attribute vector $\vec{v}$, predicate vector $\vec{u}$ satisfying $\langle \vec{v}, \vec{u} \rangle = 0$, we have $m \leftarrow \text{Decrypt}(dk_{\vec{u}}, ct')$, where $dk_{\vec{u}} \leftarrow \text{DKGen}(mpk_{ra}, msk_{ra}, \vec{u})$, $ct' \leftarrow \text{Sanitize}(mpk_{ra}, mpk_{sa}, \text{Encrypt}(mpk_{ra}, mpk_{sa}, ek_{\vec{v}, stag}, m_{sig}))$, $ek_{\vec{v}, stag} \leftarrow \text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}, stag)$, $m_{sig} \leftarrow \text{Combine}(mpk_{sa}, S, m, \{\text{PTGen}(mpk_{sa}, usk_i, m, ctag, aux)\}_{i \in S})$, and $stag = ctag$.

B. Security Model

In CSFlow, we consider the practical attacks from unauthorized sending and access attempts, as well as the collusion of unauthorized receivers. We give the security games for the no-read rule, the no-write rule, and soundness that CACE must meet to capture these attacks, respectively. Specifically, the no-read rule stipulates that any group of unauthorized receivers (described with $\vec{u}_j$) cannot obtain any information from a sender (described with $\vec{v}, stag$) if $\langle \vec{v}, \vec{u}_j \rangle = 0$ for all j . The no-write rule asserts that no collection of (potentially malicious) senders (described with $\vec{v}_i, stag_i$) can transmit any information to any group of (potentially malicious) receivers (described with $\vec{u}_j$) if $\langle \vec{v}_i, \vec{u}_j \rangle = 0$ for all i, j . The no-read and no-write rules are formulated under the “always-accept” sender policy. Moreover, the soundness requirement stipulates that a sender must only

TABLE IV
SELECTIVE SECURITY GAMES FOR NO-READ RULE AND NO-WRITE RULE OF CACE

$\text{Exp}_A^R(\lambda, \theta)$	Oracle Definition
$1. (\vec{v}_0, \vec{v}_1) \leftarrow \mathcal{A}; (pp) \leftarrow \text{GlobalSetup}(1^\lambda); \mathcal{I}_{EK} = \emptyset, \mathcal{I}_{DK} = \emptyset, \mathcal{I}_{PS} = \emptyset$ $(mpk_{ra}, msk_{ra}) \leftarrow \text{RASetsup}(pp); (mpk_{sa}, msk_{sa}, \{usk_i\}_{i \in [n]}) \leftarrow \text{SASetsup}(pp, mpk_{ra}, n, t)$ $2. (m_0, m_1) \leftarrow \mathcal{A}^{\mathcal{O}_{EK}(\cdot, \cdot), \mathcal{O}_{DK}(\cdot), \mathcal{O}_{PS}(\cdot, \cdot), \mathcal{O}_E(\cdot, \cdot)}(mpk_{ra}, mpk_{sa})$ $3. ek_{\vec{v}_0, \text{stag}} \leftarrow \text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}_0, \text{stag})$ $m_{\theta, sig_i} \leftarrow \text{PTGen}(mpk_{sa}, usk_i, m_\theta, \text{stag}, aux); m_{\theta, sig} \leftarrow \text{Combine}(mpk_{sa}, S, m_\theta, \{m_{\theta, sig_i}\}_{i \in S})$ $ct_\theta \leftarrow \text{Encrypt}(mpk_{ra}, mpk_{sa}, ek_{\vec{v}_0, \text{stag}}, m_{\theta, sig})$ $4. \theta' \leftarrow \mathcal{A}^{\mathcal{O}_{EK}(\cdot, \cdot), \mathcal{O}_{DK}(\cdot), \mathcal{O}_{PS}(\cdot, \cdot), \mathcal{O}_E(\cdot, \cdot)}(ct_\theta)$ $5. \text{return } \theta'$	$\mathcal{O}_{EK}(\vec{v}, \text{stag}) : \mathcal{I}_{EK} \leftarrow \mathcal{I}_{EK} \cup \{(\vec{v}, \text{stag})\}$ $\text{return } ek_{\vec{v}, \text{stag}} \leftarrow \text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}, \text{stag})$ $\mathcal{O}_{DK}(\vec{u}) : \mathcal{I}_{DK} \leftarrow \mathcal{I}_{DK} \cup \{\vec{u}\}$ $\text{return } dk_{\vec{u}} \leftarrow \text{DKGen}(mpk_{ra}, msk_{ra}, \vec{u})$ $\mathcal{O}_{PS}(i, m, \text{ctag}, aux) : \mathcal{I}_{PS} \leftarrow \mathcal{I}_{PS} \cup \{(i, m, \text{ctag}, aux)\}$ $\text{return } m_{sig_i} \leftarrow \text{PTGen}(mpk_{sa}, usk_i, m, \text{ctag}, aux)$ $\mathcal{O}_E(m, \vec{v}, \text{stag}) :$ $m_{sig_i} \leftarrow \text{PTGen}(mpk_{sa}, usk_i, m, \text{stag}, aux),$ $\mathcal{I}_{PS} \leftarrow \mathcal{I}_{PS} \cup \{(i, m, \text{stag}, aux)\}$ $m_{sig} \leftarrow \text{Combine}(mpk_{sa}, S, m, \{m_{sig_i}\}_{i \in S})$ $ek_{\vec{v}, \text{stag}} \leftarrow \text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}, \text{stag}),$ $\mathcal{I}_{EK} \leftarrow \mathcal{I}_{EK} \cup \{(\vec{v}, \text{stag})\}$ $\text{return } ct \leftarrow \text{Encrypt}(mpk_{ra}, mpk_{sa}, ek_{\vec{v}, \text{stag}}, m_{sig})$ $\mathcal{O}_{SE}(m, \vec{v}, \text{stag}) :$ $m_{sig_i} \leftarrow \text{PTGen}(mpk_{sa}, usk_i, m, \text{stag}, aux),$ $\mathcal{I}_{PS} \leftarrow \mathcal{I}_{PS} \cup \{(i, m, \text{stag}, aux)\}$ $m_{sig} \leftarrow \text{Combine}(mpk_{sa}, S, m, \{m_{sig_i}\}_{i \in S})$ $ek_{\vec{v}, \text{stag}} \leftarrow \text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}, \text{stag}),$ $\mathcal{I}_{EK} \leftarrow \mathcal{I}_{EK} \cup \{(\vec{v}, \text{stag})\}$ $ct \leftarrow \text{Encrypt}(mpk_{ra}, mpk_{sa}, ek_{\vec{v}, \text{stag}}, m_{sig})$ $\text{return } ct' \leftarrow \text{Sanitize}(mpk_{ra}, mpk_{sa}, ct)$
$\text{Exp}_A^W(\lambda, \theta)$	
$1. (\vec{v}', \vec{v}^*) \leftarrow \mathcal{A}; (pp) \leftarrow \text{GlobalSetup}(1^\lambda); \mathcal{I}_{EK} = \emptyset, \mathcal{I}_{DK} = \emptyset, \mathcal{I}_{PS} = \emptyset$ $(mpk_{ra}, msk_{ra}) \leftarrow \text{RASetsup}(pp); (mpk_{sa}, msk_{sa}, \{usk_i\}_{i \in [n]}) \leftarrow \text{SASetsup}(pp, mpk_{ra}, n, t)$ $ct_0 \leftarrow \mathcal{A}$ $2. \mathcal{A}^{\mathcal{O}_{EK}(\cdot, \cdot), \mathcal{O}_{DK}(\cdot), \mathcal{O}_{PS}(\cdot, \cdot), \mathcal{O}_{SE}(\cdot, \cdot)}(mpk_{ra}, mpk_{sa})$ $3. m^* \leftarrow \mathcal{M}; ek_{\vec{v}', \text{stag}} \leftarrow \text{EKGen}(mpk_{sa}, msk_{sa}, \vec{v}', \text{stag})$ $m_{sig_i}^* \leftarrow \text{PTGen}(mpk_{sa}, usk_i, m^*, \text{stag}, aux); m_{sig}^* \leftarrow \text{Combine}(mpk_{sa}, S, m^*, \{m_{sig_i}^*\}_{i \in S})$ $ct_1 \leftarrow \text{Encrypt}(mpk_{ra}, mpk_{sa}, ek_{\vec{v}', \text{stag}}, m_{sig}^*)$ $ct'_0 \leftarrow \text{Sanitize}(mpk_{ra}, mpk_{sa}, ct_0)$ $4. \theta' \leftarrow \mathcal{A}^{\mathcal{O}_{EK}(\cdot, \cdot), \mathcal{O}_{DK}(\cdot), \mathcal{O}_{PS}(\cdot, \cdot), \mathcal{O}_{SE}(\cdot, \cdot)}(ct'_0)$ $5. \text{return } \theta'$	

be permitted to encrypt messages for which they have explicit authorization. In other words, a sender who possesses encryption keys for a set of tags $\text{stag}_1, \dots, \text{stag}_Q$ cannot generate a new ciphertext ct that encrypts a message m containing the tag $ctag^*$ (regarding a certain decryption key $dk_{\vec{u}}$) if $\text{stag}_k \neq \text{ctag}^*$ for all $k \in [Q]$.

Definition 1: (No-Read Rule) We first give the selective no-read rule experiment $\text{Exp}_A^R(\lambda, \theta)$ of CACE between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ for a security parameter λ and a bit $\theta \in \{0, 1\}$ in Table IV. We say $\mathcal{A}$ wins $\text{Exp}_A^R(\lambda, \theta)$ if $\theta' = \theta$, $\forall \vec{u} \in \mathcal{I}_R, \langle \vec{v}_0, \vec{u} \rangle \neq 0, \langle \vec{v}_1, \vec{u} \rangle \neq 0$. The selective no-read rule is satisfied if for any PPT adversary $\mathcal{A}$, its advantage in winning the selective no-read game is: $|\Pr[\text{Exp}_A^R(\lambda, 0) = 0] - \Pr[\text{Exp}_A^R(\lambda, 1) = 1]| = \text{negl}(\lambda)$.

Definition 2: (No-Write Rule) We then give the selective no-write rule experiment $\text{Exp}_A^W(\lambda, \theta)$ of CACE between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ for a security parameter λ and a bit $\theta \in \{0, 1\}$ in Table IV. We say $\mathcal{A}$ wins $\text{Exp}_A^W(\lambda, \theta)$ if $\theta' = \theta$, $\forall \vec{u} \in \mathcal{I}_R, \langle \vec{v}', \vec{u} \rangle \neq 0, \langle \vec{v}^*, \vec{u} \rangle \neq 0$. The selective no-write rule is satisfied if for any PPT adversary $\mathcal{A}$, its advantage in winning the selective no-write game is: $|\Pr[\text{Exp}_A^W(\lambda, 0) = 0] - \Pr[\text{Exp}_A^W(\lambda, 1) = 1]| = \text{negl}(\lambda)$.

Definition 3. (Soundness) We then give the selective soundness experiment $\text{Exp}_A^S(\lambda)$ of CACE between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ for a security parameter λ as follows.

- **Setup:** $\mathcal{A}$ first submits a set of corrupted supervisors U with $|U| \leq t$, then $\mathcal{C}$ calls $pp \leftarrow \text{GlobalSetup}(1^\lambda)$, $(mpk_{ra}, msk_{ra}) \leftarrow \text{RASetsup}(pp)$, $(mpk_{sa}, msk_{sa}, \{usk_i\}_{i \in [n]}) \leftarrow \text{SASetsup}(pp, mpk_{ra}, n, t)$, and gives $(mpk_{ra}, mpk_{sa}, \{usk_i\}_{i \in U})$ to $\mathcal{A}$, and initializes three empty sets $\mathcal{I}_{EK}$, $\mathcal{I}_{DK}$ and $\mathcal{I}_{PS}$.
- **Query:** $\mathcal{A}$ can issue $\mathcal{O}_{PS}$ to $\mathcal{C}$ where $i \notin U$, and $\mathcal{O}_{EK}$, $\mathcal{O}_{DK}$, $\mathcal{O}_{SE}$ to $\mathcal{C}$.
- **Forgery:** $\mathcal{A}$ outputs a forgery ct^* and $\vec{u}^*$ to $\mathcal{C}$.

We say $\mathcal{A}$ wins $\text{Exp}_A^S(\lambda)$ if $\forall (\vec{v}, \text{stag}) \in \mathcal{I}_{EK}, \langle \vec{v}, \vec{u}^* \rangle = 0$ such that $|U \cup V| \leq t$, where $V = \{i : (i, m, \text{stag}, \cdot) \in \mathcal{I}_{PS}\}$, $m = \text{Decrypt}(\text{DKGen}(mpk_{ra}, msk_{ra}, \vec{u}^*), \text{Sanitize}(mpk_{ra}, mpk_{sa}, ct^*))$. The soundness is satisfied if for any PPT adversary $\mathcal{A}$, its advantage in winning the soundness experiment is: $|\Pr[\text{Exp}_A^S(\lambda) = 1] - \text{negl}(\lambda)|$.

C. Concrete Construction

We next present the construction of CSFlow, adopting the generic framework established in prior ACE schemes [18]. Specifically, we give a concrete construction from TSIG [34], EQS [35], and SIPE [23]. We also use the Schnorr NIZKP system [38] incorporated with the Fiat-Shamir heuristics [39] to produce the appropriate proof. We provide the detailed constructions of TSIG, EQS, and SIPE in Appendices A, B, and C, respectively, available online, denoted as TSIG, EQS, and SIPE.

System Setup: GA selects an asymmetric bilinear group, while RA and SA use it to initialize the system as follows.

- **CACE.GlobalSetup(1^λ).** Given the security parameter 1^λ , GA selects a bilinear group $\mathcal{BG} = (p, e, g, h, \mathbb{G}, \mathbb{H}, \mathbb{G}_T)$, where g and h are respective generators of groups $\mathbb{G}$ and $\mathbb{H}$ with its prime order p , then publishes a global public parameter $pp = \mathcal{BG}$.
- **CACE.RASetsup(pp).** RA first randomly selects $\alpha, \beta, \gamma, a_1, \dots, a_\ell, b \in \mathbb{Z}_p$, keeps the domain secret key $msk_{ra} = msk_{SIPE} = (h_0 = h^\gamma, \{h_j = h^{a_j}\}_{j \in [\ell]}, \tilde{h} = h^{\alpha\beta})$, and publishes the domain public key $mpk_{ra} = pk_{SIPE} = (g, h, g_0 = g^\gamma, \{g_j = g^{a_j}\}_{j \in [\ell]}, Y = e(g, h)^{\alpha\beta}, \tilde{g} = g^b)$.
- **CACE.SASetsup(pp, mpk_{ra}, n, t).** SA first sets $\chi = \ell + 3$, then picks random $z_1, z_2, \dots, z_\chi \in \mathbb{Z}_p$, and computes a signing key $esk = \{z_i\}_{i \in [\chi]}$, a verification key $evk = \{Z_i = h^{z_i}\}_{i \in [\chi]}$. Then, it computes $(svk, \{ssk_i, svk_i\}_{i \in [n]}) \leftarrow \text{TSIG.KeyGen}(1^\lambda, n, t)$ and defines

the effective duration *expiry* of the signature. Besides, it defines $\mathcal{L}$ as the language that for $\mathcal{V} = (mpk_{ra}, mpk_{sa}, c, \bar{C}, \bar{E}, \mu', \mu'')$ and $\mathcal{W} = (m, (\zeta, \tau), stag)$ where ζ and τ are the randomness in c and $stag$ is the tag in the encryption key, we have $\mathcal{R}(\mathcal{V}, \mathcal{W}) = 1$ iff $1 = \text{EQS.Verify}(evk, \mathcal{K}' = (g^\zeta, g_0^\zeta, (\tilde{g}^{v_1} g_1)^\zeta, \dots, (\tilde{g}^{v_\ell} g_\ell)^\zeta, g^{stag \cdot \zeta}), \mu') \wedge 1 = \text{EQS.Verify}(evk, \mathcal{K}'' = (g^\tau, g_0^\tau, (\tilde{g}^{v_1} g_1)^\tau, \dots, (\tilde{g}^{v_\ell} g_\ell)^\tau, g^{stag \cdot \tau}), \mu'') \wedge c = \text{SIPE.Encrypt}(pk_{\text{SIPE}}, m, \vec{v})$. At last, it computes $crs \leftarrow \text{NIZKP.Gen}(1^\lambda, \mathcal{L})$ and publishes the domain public key $mpk_{sa} = (svk, \{svk_i\}_{i \in [n]}, evk, crs, expiry)$, keeps the domain secret key $msk_{sa} = esk$ and sends $usk_i = ssk_i$ to the i -th supervisor in a secure channel.

Key Generation: For each sender, SA issues a fine-grained sender policy $(\vec{v}, stag)$ and creates the encryption key. As for each receiver, RA issues a decryption key associated with $\vec{u}$.

- **CACE.EKGen**($mpk_{sa}, msk_{sa}, \vec{v}, stag$). SA first extracts a vector $\mathcal{K} = (K, K_0, K_1, \dots, K_\ell, \bar{K}) = (g, g_0, \tilde{g}^{v_1} g_1, \dots, \tilde{g}^{v_\ell} g_\ell, g^{stag})$ from the attribute vector $\vec{v}$ and tag $stag$. Then it randomly chooses $r \in \mathbb{Z}_p$, and follows **EQS.Sign**($esk, \mathcal{K}$) to compute a signature $\mu = (Y_0 = K^{z_1 r} \cdot K_0^{z_2 r} \cdot \prod_{i=1}^\ell K_i^{z_{i+2} r} \cdot \bar{K}^{z_x r}, Y_1 = g^{1/r}, Y_2 = h^{1/r})$. Finally, SA dispatches the encryption key $ek_{\vec{v}, stag} = (\mathcal{K}, \mu)$ together with $\vec{v}, stag$ to the sender in a secure channel.
- **CACE.DKGen**($mpk_{ra}, msk_{ra}, \vec{u}$). RA randomly selects $r_0, r_1 \in \mathbb{Z}_p$, follows **SIPE.KeyGen**($pk_{\text{SIPE}}, mk_{\text{SIPE}}, \vec{u}$) to compute the decryption key $dk_{\vec{u}} = (D_0 = h^{r_0}, D_1 = h^{r_1}, D_2 = \tilde{h} \cdot h_0^{r_0} \prod_{j=1}^\ell (h_j)^{u_j r_1})$ and sends it to the receiver in a secure channel.

Data Tagging. When a sender intends to deliver a message m , she/he first chooses a random $\zeta \in \mathbb{Z}_p$, and sends $(m, aux = \zeta)$ to the supervisors securely. After receiving the request, the supervisors specify its content tag $ctag$ and generate the partial tag-signed messages as follows.

- **CACE.PTGen**($mpk_{sa}, usk_i, m, ctag, aux$). The i -th supervisor computes $C = m \cdot Y^\zeta, \bar{C} = g^{ctag \cdot \zeta}$ and generates a partial signature $\delta_i \leftarrow \text{TSIG.PSign}(usk_i, C || \bar{C} || time)$, where $time$ is the current timestamp. Finally, it sends $m_{sig_i} = (ctag, \zeta, time, \delta_i)$ to the sender securely.

After getting $t + 1$ partial tag-signed messages (we let S be the index set), the sender combines them into the final tag-signed message m_{sig} as follows.

- **CACE.Combine**($mpk_{sa}, S, m, \{m_{sig_i}\}_{i \in S}$). The sender first parses m_{sig_i} into $(ctag, \zeta, time, \delta_i)$, verifies it and computes the signature $\delta \leftarrow \text{TSIG.Comb}(S, C || \bar{C} || time, \{(svk_i, \delta_i)\}_{i \in S})$. Finally, the sender gets the tag-signed message $m_{sig} = (m, \zeta, time, \delta)$.

Data Sending: After obtaining the tag-signed message m_{sig} , the sender first checks whether the signature is timed out by verifying if $|time - time_0| \leq expiry$, where $time_0$ is the current timestamp. If the check fails, the sender re-requests the tag-signed message from the supervisors. Finally, the sender uses the encryption key $ek_{\vec{v}, stag}$ to create the original ciphertext ct as follows.

- **CACE.Encrypt**($mpk_{ra}, mpk_{sa}, ek_{\vec{v}, stag}, m_{sig}$). The sender first parses m_{sig} into $(m, \zeta, time, \delta)$ and chooses a random $\tau \in \mathbb{Z}_p$. Then it computes $\bar{C} = \bar{K}^\zeta = g^{stag \cdot \zeta}, \bar{E}$

$= \bar{K}^\tau = g^{stag \cdot \tau}$, and follows SIPE to compute

$$c = (C = m \cdot Y^\zeta, \bar{C} = K^\zeta, C_0 = K_0^\zeta, \{C_j = K_j^\zeta\}_{j \in [\ell]}, E = Y^\tau, \bar{E} = K^\tau, E_0 = K_0^\tau, \{E_j = K_j^\tau\}_{j \in [\ell]}).$$

Then it computes the proofs for the sending operation.

- It picks a random $\psi_1 \in \mathbb{Z}_p$, and follows **EQS.ChgRep**($evk, \mathcal{K}, \mu, \zeta$) to compute an adapted signature $\mu' = (Y'_0 = (Y_0)^{\zeta \psi_1}, Y'_1 = (Y_1)^{1/\psi_1}, Y'_2 = (Y_2)^{1/\psi_1})$ on $[\mathcal{K}'] = [\mathcal{K}^\zeta]$.
- It picks a random $\psi_2 \in \mathbb{Z}_p$, and follows **EQS.ChgRep**($evk, \mathcal{K}, \mu, \tau$) to compute an adapted signature $\mu'' = (Y''_0 = (Y_0)^{\tau \psi_2}, Y''_1 = (Y_1)^{1/\psi_2}, Y''_2 = (Y_2)^{1/\psi_2})$ on $[\mathcal{K}'] = [\mathcal{K}^\tau]$.
- Let $\mathcal{V} = (mpk_{ra}, mpk_{sa}, c, \bar{C}, \bar{E}, \mu', \mu'')$ be the statement and $\mathcal{W} = (m, (\zeta, \tau), stag)$ be the witness. It outputs a proof $\pi \leftarrow \text{NIZKP.Prove}(crs, \mathcal{V}, \mathcal{W})$.

Finally, the sender constructs the original ciphertext $ct = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$, and transmits it to the connected edge node. In detail, the Fiat-Shamir heuristic is adopted to generate Schnorr-style proofs for the following equations.

$$\text{PoK} \left\{ (\mathcal{W}) : \begin{array}{l} \wedge m \cdot Y^\zeta = C \wedge g^\zeta = \bar{C} \wedge g^{stag \cdot \zeta} = \bar{C} \\ \wedge Y^\tau = E \wedge g^\tau = \bar{E} \wedge g^{stag \cdot \tau} = \bar{E} \end{array} \right. \quad (1)$$

Ciphertext Sanitization: After receiving the original ciphertext ct from the sender, the edge node performs the following sanitization process on the ciphertext.

- **CACE.Sanitize**(mpk_{ra}, mpk_{sa}, ct). The edge node first checks whether the ciphertext is legitimate as follows.
 - Check the integrity of the ciphertext components, by verifying if $\text{NIZKP.Verify}(crs, \mathcal{V} = (mpk_{ra}, mpk_{sa}, c, \bar{C}, \bar{E}, \mu', \mu''), \pi) = 1$.
 - Check if the ciphertext ct is generated by an authorized fine-grained sender policy, namely $(\vec{v}, stag)$, by verifying the EQS signatures μ' and μ'' , i.e. whether $e(\bar{C}, Z_1) \cdot e(C_0, Z_2) \cdot \prod_{i=1}^\ell e(C_i, Z_{i+2}) \cdot e(\bar{C}, Z_x) = e(Y'_0, Y'_2) \cdot e(Y'_1, h) = e(g, Y'_2)$, and $e(\bar{E}, Z_1) \cdot e(E_0, Z_2) \cdot \prod_{i=1}^\ell e(E_i, Z_{i+2}) \cdot e(\bar{E}, Z_x) = e(Y''_0, Y''_2) \cdot e(Y''_1, h) = e(g, Y''_2)$ hold.
 - Check if the content tag in ct is also certified by the supervisors, by verifying if $\text{TSIG.Verify}(svk, C || \bar{C} || time, \delta) = 1$ and $|time - time_1| \leq expiry$, where $time_1$ is the current timestamp.

If any check fails, the edge node drops ct . Otherwise, it further randomly picks $s \in \mathbb{Z}_p$ and computes $c' =$

$$(C' = C \cdot E^s = m \cdot Y^{\zeta + \tau s}, \bar{C}' = \bar{C} \cdot \bar{E}^s = g^{\zeta + \tau s}, C'_0 = C_0 \cdot E_0^s = g_0^{\zeta + \tau s}, \{C'_j = C_j \cdot E_j^s = (\tilde{g}^{v_j} g_j)^{\zeta + \tau s}\}_{j \in [\ell]}),$$

and sends the sanitized ciphertext $ct' = c'$ to the cloud.

Data Receiving: After a receiver obtains the sanitized ciphertext ct' from the edge node, she/he uses the decryption key $dk_{\vec{u}}$ to recover the message as follows.

- **CACE.Decrypt**($dk_{\vec{u}}, ct'$). If $\langle \vec{v}, \vec{u} \rangle = 0$, the receiver can efficiently runs **SIPE.Decrypt**($dk_{\vec{u}}, ct'$) to compute $A = e(\bar{C}', D_2) / (e(C'_0, D_0) \cdot e(\prod_{j=1}^\ell (C'_j)^{u_j}, D_1)) =$

$e(g, h)^{\alpha\beta(\zeta+\tau s)}$ with 3 pairing computations, and then recover the message $m = C'/A$.

VI. SECURITY ANALYSIS

In this section, we provide security proofs for the selective no-read, selective no-write rule and the soundness of CACE, and security analysis of CSFlow based on these proofs.

A. Selective No-Read Rule of CACE

Theorem 1: CACE fulfills the selective no-read rule provided that NIZKP is zero-knowledge, EQS provides perfect adaptation, and SIPE provides IND-sAH-CPA security.

Proof: We construct the following sequence of hybrids to exhibit that $\text{Exp}_A^R(\lambda, \theta)$ is computationally indistinguishable for the adversary.

- $H_{b,0}$ ($b \in \{0, 1\}$): Defined as the selective no-read game $\text{Exp}_A^R(\lambda, \theta)$ with θ fixed to b .
- $H_{b,1}$ ($b \in \{0, 1\}$): Defined as $H_{b,0}$ except that μ' and μ'' in the challenge ciphertext are freshly computed.
- $H_{b,2}$ ($b \in \{0, 1\}$): Defined as $H_{b,1}$ except that crs in mpk_{sa} and π in the challenge ciphertext are simulated.

Firstly, according to the perfect adaptation property of EQS, we have $ct = (c, \bar{C}, \bar{E}, \text{EQS.ChgRep}(evk, \mathcal{K}, \mu, \zeta), \text{EQS.ChgRep}(evk, \mathcal{K}, \mu, \tau), \pi, \delta, time) = (c, \bar{C}, \bar{E}, \text{EQS.Sign}(esk, \mathcal{K}' = \mathcal{K}^\zeta), \text{EQS.Sign}(esk, \mathcal{K}'' = \mathcal{K}^\tau), \pi, \delta, time)$, hence $H_{b,1}$ is indistinguishable from $H_{b,0}$. Secondly, according to the zero-knowledge property of the NIZKP system, $H_{b,2}$ is indistinguishable from $H_{b,1}$. Thirdly, the indistinguishability between $H_{0,2}$ and $H_{1,2}$ follows from the IND-sAH-CPA security of SIPE. If an adversary $\mathcal{A}$ can differentiate between them, we can build an algorithm $\mathcal{B}$ that is able to break the IND-sAH-CPA security of SIPE. Specifically, $\mathcal{B}$ uses $\vec{v}_0$ and $\vec{v}_1$ as its challenge attribute vectors and sends them to its challenger. When $\mathcal{A}$ submits $(m_0, m_1, stag)$ as challenge, $\mathcal{B}$ forwards (m_0, m_1) to its challenger and gets the response c . Next, it computes $\bar{C} = \bar{C}^{stag}$, $\bar{E} = \bar{E}^{stag}$, and $\mathcal{K}', \mathcal{K}''$ from $(c, \bar{C}, \bar{E})$, and also computes μ', μ'' from them. Finally, $\mathcal{B}$ creates a signature δ and a simulated π , and returns a CACE challenge ciphertext $ct = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$ to $\mathcal{A}$. Thus, $\mathcal{B}$ can perfectly simulate either $H_{0,2}$ or $H_{1,2}$, and the IND-sAH-CPA security of SIPE ensures that $H_{0,2}$ is indistinguishable from $H_{1,2}$. We can conclude that the construction of CACE satisfies the selective no-read rule. ■

B. Selective No-Write Rule of CACE

Theorem 2: CACE fulfills the selective no-write rule provided that the NIZKP system is knowledge-extractable and perfectly sound, EQS provides EUF-CMA security, and SIPE provides weak sanitizability.

Proof: We construct the following sequence of hybrids to exhibit that $\text{Exp}_A^W(\lambda, \theta)$ is computationally indistinguishable for the adversary.

- $H_{b,0}$ ($b \in \{0, 1\}$): Defined as the selective no-write game $\text{Exp}_A^W(\lambda, \theta)$ with θ fixed to b .
- $H_{b,1}$ ($b \in \{0, 1\}$): Defined as $H_{b,0}$ with the exception that crs is obtained from the knowledge extractor. Therefore,

when $\mathcal{A}$ submits $ct_0 = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$, we can execute the second part of the extractor to extract the witness $\mathcal{W} = (m, (\zeta, \tau), stag)$ from π .

Firstly, according to the first property of knowledge extraction in the proof system, $H_{b,0}$ and $H_{b,1}$ cannot be distinguished from each other. Further, we categorize the run of $H_{b,1}$ into the following four types:

- T_1 : $\mathcal{A}$ submits $ct_0 = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$, where $\text{NIZKP.Verify}(crs, \mathcal{V}, \pi) = 0$ and $\mathcal{V} = (mpk_{ra}, mpk_{sa}, c, \bar{C}, \bar{E}, \mu', \mu'')$.
- T_2 : $\mathcal{A}$ submits $ct_0 = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$, where $\text{NIZKP.Verify}(crs, \mathcal{V}, \pi) = 1 \wedge 1 = \text{TSIG.Verify}(svk, C||\bar{C}||time, \delta)$ and $1 \neq \text{EQS.Verify}(evk, \mathcal{K}', \mu') \vee 1 \neq \text{EQS.Verify}(evk, \mathcal{K}'', \mu'') \vee c \neq \text{SIPE.Encrypt}(pk_{\text{SIPE}}, m, \vec{v})$.
- T_3 : $\mathcal{A}$ submits $ct_0 = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$, where $\text{NIZKP.Verify}(crs, \mathcal{V}, \pi) = 1 \wedge 1 = \text{TSIG.Verify}(svk, C||\bar{C}||time, \delta)$ and $1 = \text{EQS.Verify}(evk, \mathcal{K}', \mu') \wedge 1 = \text{EQS.Verify}(evk, \mathcal{K}'', \mu'') \wedge c = \text{SIPE.Encrypt}(pk_{\text{SIPE}}, m, \vec{v})$, and for all $\mathcal{K}$ obtained via the encryption key query to $\mathcal{C}$ satisfying $[\mathcal{K}] \neq [\mathcal{K}']$ and $[\mathcal{K}] \neq [\mathcal{K}']$.
- T_4 : Defined as T_4 with the exception that there exists $\mathcal{K}$ obtained via the encryption key query to $\mathcal{C}$ satisfying $[\mathcal{K}] = [\mathcal{K}'] = [\mathcal{K}']$.

We contend that these four types constitute a partition of the coin space in the experiment, and it is impossible for $\mathcal{A}$ to win $H_{b,1}$ in a non-negligible probability.

- If case T_1 occurs, $\mathcal{A}$ loses the game since the CACE.Sanitize procedure aborts.
- If case T_2 occurs, $\mathcal{A}$ breaks the second property of knowledge extraction security of the NIZKP system. In other words, $\mathcal{A}$ outputs a valid proof π for statement $\mathcal{V} = (mpk_{ra}, mpk_{sa}, c, \bar{C}, \bar{E}, \mu', \mu'')$, but the extractor E_2 is unable to extract a witness $\mathcal{W}$ where $(\mathcal{V}, \mathcal{W}) \in \mathcal{R}$.
- In case T_3 , we can regard it as a forgery attack on the underlying EQS scheme. Especially, we build a PPT algorithm $\mathcal{B}$ that simulates $H_{b,1}$ for $\mathcal{A}$, which submits signing queries to its challenger in response to the encryption key queries of $\mathcal{A}$. When $\mathcal{A}$ submits ct_0 , $\mathcal{B}$ uses the knowledge extractor to extract $\mathcal{W} = (m, (\zeta, \tau), stag)$ from π and computes $\mathcal{K}^*$ from $\mathcal{K}'$ and $\mathcal{K}''$, where $[\mathcal{K}^*] = [\mathcal{K}'] = [\mathcal{K}']$. Since $\mathcal{B}$ did not query the message $\mathcal{K}^* = (g, g_0, \tilde{g}^{y_1} g_1, \dots, \tilde{g}^{y_\ell} g_\ell, g^{stag})$ corresponding to $(\vec{v}, stag)$ to its signing oracle that returns a signature on $[\mathcal{K}^*]$, $\mathcal{B}$ outputs two valid forgeries $(\mathcal{K}', \mu')$ and $(\mathcal{K}'', \mu'')$.
- In case T_4 , the two hybrid games $H_{0,1}$ and $H_{1,1}$ are distinguishable, where c' is from m in $H_{0,1}$, and from m^* in $H_{1,1}$. If $\mathcal{A}$ is able to differentiate between these two games, then we could design an algorithm $\mathcal{B}$ compromising the weak sanitizability of SIPE. Specifically, $\mathcal{B}$ uses $\vec{v}'$ and $\vec{v}^*$ as its challenge attribute vectors and sends them to its challenger. After $\mathcal{A}$ submits ct_0 , $\mathcal{B}$ extracts $(m, (\zeta, \tau), stag)$ from π in the ct_0 , chooses a random m^* , and submits (m, m^*) to its weak sanitizability challenger. Finally, $\mathcal{B}$ forwards the sanitized ciphertext c' to $\mathcal{A}$ as a challenge. The weak sanitizability of SIPE ensures that $H_{0,1}$ is indistinguishable from $H_{1,1}$.

Since the NIZKP system is knowledge extractable and perfectly sound, EQS provides EUF-CMA security, and SIPE is weakly sanitizable, we conclude this proof. ■

C. Soundness of CACE

Theorem 3: CACE is sound provided that the NIZKP system is knowledge-extractable and perfectly sound, the underlying TSIG provides EUF-CMA security.

Proof: To establish our proof, we construct the following sequence of hybrid experiments.

- H_0 : Defined as the soundness game $\text{Exp}_A^S(\lambda)$.
- H_1 : Defined as H_0 except that crs is obtained from the knowledge extractor. After $\mathcal{A}$ submits a challenge ciphertext $ct^* = (c, \bar{C}, \bar{E}, \mu', \mu'', \pi, \delta, time)$ and a predicate vector $\bar{u}^*$, we run the second part of the extractor to extract the witness $\mathcal{W} = (m, (\zeta, \tau), stag)$ from π .

Firstly, according to the zero-knowledge property of the NIZKP system, H_1 is indistinguishable from H_0 . Secondly, if $\mathcal{A}$ wins in H_1 , we could build an algorithm $\mathcal{B}$ that is able to destroy the EUF-CMA security of TSIG. Specifically, when receiving a partial tag-signed message generation query $(i, m, ctag, aux = \zeta)$ from $\mathcal{A}$, $\mathcal{B}$ computes $C = m \cdot Y^\zeta$, $\bar{C} = g^{ctag \cdot \zeta}$. Then $\mathcal{B}$ queries $C || \bar{C} || time$ to TSIG challenger in response to the partial tag-signed message generation queries of $\mathcal{A}$. The challenger outputs 1 only if $m \neq \perp$, δ is a valid signature on $C || \bar{C} || time$, where $C = m \cdot Y^\zeta$, $\bar{C} = g^{ctag \cdot \zeta}$. Since $\mathcal{A}$ gets at most t partial tag-signed messages about $(\cdot, m, ctag, \zeta)$, $\mathcal{B}$ outputs a forgery $(C || \bar{C} || time, \delta)$ for TSIG. As a result, we can conclude that the construction of CACE is sound. ■

D. Security Analysis of CSFlow

CSFlow is instantiated via CACE, which complies with the selective no-read rule, the selective no-write rule, and soundness. Subsequently, we delve into the potential attacks on the CSFlow system and illustrate the methods to counteract these attacks. First, the adversaries possessing a legitimate encryption key for $(\vec{v}, stag)$ may collude to send a message m containing a tag $ctag$, where $ctag \neq stag$. Even if an adversary corrupts t supervisors, she/he is unable to create a valid original ciphertext. This scenario can be regarded as an attack on the soundness of CACE.

Second, the no-read rule of CACE ensures that the adversaries cannot collude to decrypt a ciphertext. Furthermore, adversaries might fabricate a spurious encryption key to disseminate sensitive data to unauthorized recipients or derive a legitimate new ciphertext from the collected ciphertexts. Essentially, this attack amounts to a violation of the no-write rule of CACE. In addition, edge nodes, cloud servers, and external adversaries may attempt to access the shared data and receivers' information. This kind of attack can be essentially regarded as an infringement of the no-read rule of CACE.

VII. PERFORMANCE EVALUATION

In this section, we compare CACE with the state-of-the-art schemes Sedaghat's LSSS CD-ABACE [17], Wang's LSSS

ABACE [18], and CD-IPACE [23], in terms of the computational overhead and storage overhead. In addition, CACE chooses the Boneh–Lynn–Shacham (BLS) TSIG scheme [34] to bind the data with its tag.

A. Theoretical Analysis

Tables V and VI present a comprehensive analysis of computational and storage overhead. In Table V, we primarily focus on the time-consuming calculations, namely, the exponentiation operation and bilinear pairings. Specifically, e_0 , e_1 , and e_t represent the running times of an exponentiation operation in the groups $\mathbb{G}$, $\mathbb{H}$, and $\mathbb{G}_T$, respectively. Let e_p denote the running time of a pairing operation, ℓ be the number of rows of the LSSS matrix (the length of the attribute vector) in the encryption key generation phase, n be the number of attributes (the length of the predicate vector) in the decryption key generation phase, e_q/e_v be the time required to prove/verify an equation using NIZKP, s_v be the verification time of the TSIG. In Table VI, the storage costs of single-group elements in different groups are presented, $|\mathbb{G}|$, $|\mathbb{H}|$, $|\mathbb{G}_T|$ denote the storage cost of a single group element in respective $\mathbb{G}$, $\mathbb{H}$, $\mathbb{G}_T$. Let $|\pi|$ be the storage cost for using the NIZKP, $|\delta|$ be the size of the signature.

From Tables V and VI, we can make a few notable observations as follows. First, ABACE, CD-IPACE, and CACE perform approximately ℓe_1 operations to generate a decryption key, while CD-IPACE and CACE get a constant-size decryption key. Second, CACE and CD-IPACE have an advantage over CD-ABACE and ABACE in the encryption and sanitization phase, because they need to prove and verify fewer equations using time-consuming NIZKP. Moreover, the size of sanitized ciphertexts is less than that of original ciphertexts in all four schemes, which is directly related to the cloud storage fee. Similarly, CACE and CD-IPACE share the same computational time in the decryption phase, due to their reliance on the same SIPE technique, which only needs three pairing operations to decrypt a ciphertext, outperforming ABACE and CD-ABACE. Next, we will verify our results through experimental evaluation.

B. Experimental Analysis

We implemented these ACE schemes using the JPBC framework [40] and the BN254 curve for pairings. We adopt the edge network topology of Fig. 3 and perform the experimental simulation on a set of devices equipped with an Intel Core i7-9750H CPU and 16 GB RAM, where the edge node acts as a gateway to control the traffic flowing to the cloud server. First, we estimate the micro benchmarks of main group operations on the BN254 curve. In detail, it costs 4.79 s, 7.75 s, 6.36 s to compute an exponentiation operation in the group $\mathbb{G}$, $\mathbb{H}$, $\mathbb{G}_T$, and 59.63 s to execute a pairing operation. Then, we range the number ℓ of user attributes in the policy from 2 to 20 and compare the computational time and storage cost. Concretely, we use access structure in the form of "Attr₁ AND Attr₂ AND $\dots$ AND Attr _{ℓ} " as in FAME [7] because all ℓ attributes are then required for decryption.

Fig. 5 presents the running time of EKGen, DKGen, Encrypt, Sanitize, and Decrypt algorithms in those schemes. From

TABLE V
COMPARISON OF COMPUTATIONAL COST

Schemes	Key Generation	Encryption	Sanitization	Decryption
CD-ABACE [17]	$5\ell e_0 + \ell e_1$ (Encryption Key) $(n+1)e_0 + e_1$ (Decryption Key)	$(5\ell+1)e_0 + \ell e_1 + e_t + 2\ell e_q$	$2\ell e_0 + (5\ell+1)e_p + 2\ell e_v$	$(2\ell+1)e_p + \ell e_t$
ABACE [18]	$(12\ell+6)e_0 + (2\ell+2)e_1$ (Encryption Key) $(n+2)e_1$ (Decryption Key)	$(12\ell+6)e_0 + (2\ell+1)e_1 + 3e_t + (4\ell+4)e_q$	$(2\ell+1)e_0 + e_t + (10\ell+6)e_p + (4\ell+4)e_v$	$(2\ell+1)e_p + \ell e_t$
CD-IPACE [23]	$\ell e_0 + (2\ell+1)e_1$ (Encryption Key) $(n+3)e_1$ (Decryption Key)	$(2\ell+2)e_0 + 2e_t + 9e_q$	$(\ell+1)e_0 + e_t + 9e_v$	$\ell e_0 + 3e_p$
CACE	$(2\ell+5)e_0 + e_1$ (Encryption Key) $(n+3)e_1$ (Decryption Key)	$(2\ell+10)e_0 + 2e_1 + e_t + 6e_q$	$(\ell+1)e_0 + e_t + (2\ell+6)e_p + 6e_v + s_v$	$\ell e_0 + 3e_p$

TABLE VI
COMPARISON OF STORAGE COST

Schemes	Encryption Key	Decryption Key	Original Ciphertext	Sanitized Ciphertext
CD-ABACE [17]	$(3\ell+1) G + H $	$(\ell+1) G + H $	$(4\ell+2) G + 2 H + G_T + \pi $	$(2\ell+1) G + H + G_T $
ABACE [18]	$(6\ell+4) G + H + G_T $	$(\ell+2) H $	$(10\ell+6) G + 2 H + 3 G_T + \pi $	$(2\ell+1) G + G_T $
CD-IPACE [23]	$\ell G + (\ell+2) H + \ell Z_p $	$3 H $	$(2\ell+2) G + 2 G_T + \pi $	$(\ell+2) G + G_T $
CACE	$(\ell+5) G + H $	$3 H $	$(2\ell+10) G + 2 H + 2 G_T + \pi + \delta $	$(\ell+2) G + G_T $

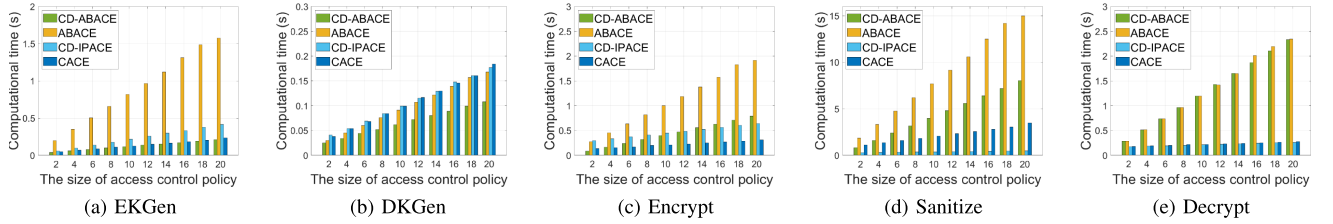


Fig. 5. Computational time of EKGen, DKGen, Encrypt, Sanitize, Decrypt algorithms.

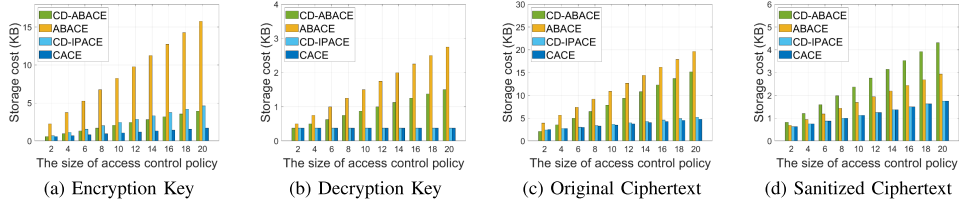


Fig. 6. Storage cost of encryption key, decryption key, original ciphertext, and sanitized ciphertext.

Fig. 5(a), we can observe that even though the encryption key generation time of each work grows linearly with ℓ , CACE and CD-IPACE have an advantage over CD-ABACE and ABACE, with the reason that CACE has fewer time-consuming exponentiations in the group $\mathbb{H}$. Especially, it costs 0.23s in CACE when $\ell = 20$. As seen in Fig. 5(b), although CACE is less efficient than CD-ABACE in the decryption key generation phase, it is comparable to ABACE and CD-IPACE. This is because they have more time-consuming exponentiations in the group $\mathbb{H}$ than CD-ABACE. In terms of encryption, we optimize CD-IPACE inspired by ABACE, hence CACE is more time efficient. As shown in Fig. 5(c), the encryption time in the four schemes is proportional to ℓ , but CACE has the best performance, and it increases from 0.13 s to 0.31 s in CACE. In the sanitization phase, CACE takes more computational time than CD-IPACE, but much less than CD-ABACE and ABACE, as depicted in Fig. 5(d). Especially, it costs 1.60s in CD-ABACE, 3.32 s in ABACE, 0.32s in CD-IPACE, and 1.36 s in CACE to sanitize an original ciphertext when $\ell = 4$. In the decryption phase, CACE only needs three pairing operations to decrypt a ciphertext, hence

it has a significant advantage over CD-ABACE and ABACE, as shown in Fig. 5(e). In particular, it takes 0.26s in CACE and CD-IPACE, 2.34s in ABACE and CD-ABACE to decrypt a sanitized ciphertext when $\ell = 20$.

Fig. 6 displays the storage cost of encryption key, decryption key, original ciphertext, and sanitized ciphertext in those schemes. From Fig. 6(a), we can see that the size of the encryption key in CACE increases narrowly with ℓ , and outperforms CD-ABACE, ABACE, and CD-IPACE. Specifically, it increases from 0.56 KB to 1.69 KB in CACE. Regarding the size of the decryption key, we can observe that it is constant in CACE and CD-IPACE, and is proportional to ℓ in CD-ABACE and CD-ABACE as described in Fig. 6(b). When $\ell = 20$, the size of the decryption key is 0.34 KB in CACE and CD-IPACE, 1.5 KB in CD-ABACE, and 2.75 KB in ABACE. Observing Fig. 6(c) and Fig. 6(d), it is obvious that the size of the sanitized ciphertext is smaller than that of the original ciphertexts. Taking CACE as an example, when $\ell = 10$, the size of the original ciphertexts is 3.5 KB, while that of the sanitized ciphertexts is 1.13 KB. Generally speaking, lower storage costs for ciphertext

TABLE VII
THE IMPACT OF THRESHOLD t

t	Computational time (ms)			Storage cost (KB)	
	SASetup	PTGen	Combine	m_{sig_i}	m_{sig}
1	108	100	9	0.89	0.54
3	110	199	16	1.79	0.54
5	112	298	22	2.68	0.54
7	114	401	29	3.58	0.54
9	116	502	36	4.47	0.54

and decryption keys tend to lead to more efficient decryption. Specifically, in our CACE, we have minimized the storage overhead for decryption keys and sanitized ciphertext, thereby achieving a more efficient decryption process than ABACE and CD-ABACE.

In addition, we further change the threshold t from 1 to 9 with an interval of 2 to evaluate its impact on the computational time and storage cost. As seen in Table VII, the setup of the SA increases from 108ms to 116ms, which demonstrates that decentralized supervisors have little impact on the system setup. Furthermore, it takes 9ms to compute a partial tag-signed message m_{sig_i} that is 0.89 KB in size. The computational time to aggregate the partial tag-signed messages into the tag-signed message m_{sig} is proportional to t . It increases from 9 to 36 ms as t increases from 1 to 9. To sum up, the theoretical analysis and experimental analysis demonstrate that our CACE has efficient encryption, sanitization, and decryption operations compared to CD-ABACE and ABACE, and simultaneously achieves fine-grained sender policies without incurring heavy computational and storage overhead compared to CD-IPACE.

C. Case Studies

CSFlow can be utilized for secure flow control in EMR sharing, which serves as a representative real-world case study where data confidentiality, authenticity, and regulated data usage are required under GDPR. In this deployment, CSFlow operates within a healthcare consortium composed of multiple hospitals, authorized data consumers (e.g., research institutes or insurance agencies), edge service providers, and public cloud platforms, forming a typical cloud-edge architecture. Hospitals act as data senders, while authorized institutions act as receivers. The global and domain authorities correspond to consortium-level system administrators for system initialization and key management. Supervisors represent real-world data governance roles, such as data protection officers or hospital compliance committees, who are already responsible for approving data sharing and ensuring regulatory compliance. All underlying cryptographic operations, such as key generation, tag-signed message generation, encryption, sanitization, and decryption, are performed locally, as illustrated in Fig. 4. Before sharing EMRs, hospitals must obtain sufficient supervisor approvals in the form of tag-signed messages, ensuring that the data to be shared complies with predefined policies. At the edge, ciphertext sanitization is performed to verify that embedded content tags and receiver attributes conform to the sender's authorized policy, thereby enforcing data flow control and filtering out unauthorized or malformed ciphertexts. This case study demonstrates

that CSFlow can be seamlessly integrated into existing cloud-edge infrastructures, such as public cloud platforms combined with organization-operated edge servers, while ensuring that all sensitive cryptographic operations and policy enforcement are executed locally. Beyond EMR sharing, CSFlow can be naturally extended to other cross-domain scenarios, such as industrial IoT data collaboration and cross-institutional financial data exchange.

- *Case 1:* The power grid company holds diverse data types, including users' personal information, bank card details, and electricity consumption records. To analyze regional power usage patterns and optimize public grid dispatch strategies, Alice is tasked with aggregating residential electricity data m for submission to the city's energy management centre. However, prior to transmission, she must obtain data authorization m_{sig} from the supervisor. As required by legal regulations, she may only send anonymized regional-level electricity consumption data to the city energy management centre.
- *Case 2:* Multiple banks wish to train a cross-bank suspicious transaction monitoring model. After receiving both transmission authorization $ek_{\bar{g}}$ and data authorization m_{sig} over the parameters m , she sends the encrypted parameters ct to the edge node. The edge node verifies the legitimacy of the ciphertext sent by Alice; if invalid, the data is discarded; otherwise, it is forwarded to the cloud server. The authorized server ultimately aggregates all parameters to derive the global model, which is then distributed to each bank.

By explicitly incorporating real-world organizational roles, authorization workflows, and deployment environments, CSFlow bridges the gap between theoretical design and practical, regulation-aware flow control systems.

VIII. CONCLUSION

In this paper, we present CSFlow, a content-aware secure flow control system for fine-grained encrypted data sharing in cloud-edge, which employs edge nodes to monitor all encrypted data outsourced to the cloud server. To resist malicious senders, we introduce supervisors to authenticate the data content and impose content-aware sanitization on the encrypted data from the senders. We instantiate it by introducing a new cryptographic primitive named CACE, a cross-domain ACE with fine-grained sender policies. We construct CACE from a SIPE scheme, an EQS scheme, a TSIG scheme, and the NIZKP system. We formally prove its security based on these building blocks and demonstrate its reasonable efficiency through algorithm simulation. The experimental results have shown that CSFlow is secure and practical for cross-domain data flow control in cloud-edge computing.

REFERENCES

- [1] Y. Yang, X. Liu, R. H. Deng, and Y. Li, "Lightweight sharable and traceable secure mobile health system," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 1, pp. 78–91, Jan./Feb. 2020.
- [2] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2005, pp. 457–473.

- [3] S. Xu, G. Yang, Y. Mu, and R. H. Deng, "Secure fine-grained access control and data sharing for dynamic groups in the cloud," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 8, pp. 2101–2113, Aug. 2018.
- [4] Q. Huang, Y. Yang, W. Yue, and Y. He, "Secure data group sharing and conditional dissemination with multi-owner in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 9, no. 4, pp. 1607–1618, Oct.–Dec. 2021.
- [5] B. Waters, "Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization," in *Proc. Int. Workshop Public Key Cryptogr.*, 2011, pp. 53–70.
- [6] Y. Rouselakis and B. Waters, "Practical constructions and new proof methods for large universe attribute-based encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2013, pp. 463–474.
- [7] S. Agrawal and M. Chase, "FAME: Fast attribute-based message encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 665–682.
- [8] Y. S. Rao, "A secure and efficient ciphertext-policy attribute-based sign-cryption for personal health records sharing in cloud computing," *Future Gener. Comput. Syst.*, vol. 67, pp. 133–151, Feb. 2017.
- [9] G. Ateniese, D. Francati, D. Nufiez, and D. Venturi, "Match me if you can: Matchmaking encryption and its applications," *J. Cryptol.*, vol. 34, no. 3, Apr. 2021, Art. no. 16.
- [10] S. Xu et al., "Match in my way: Fine-grained bilateral access control for secure cloud-fog computing," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1064–1077, Mar./Apr. 2020.
- [11] J. Sun, G. Xu, T. Zhang, X. Tang, M. Alazab, and R. H. Deng, "Privacy-aware and security-enhanced efficient matchmaking encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4345–4360, 2023.
- [12] I. Damgård, H. Haagh, and C. Orlandi, "Access control encryption: Enforcing information flow with cryptography," in *Proc. Theory Cryptogr.*, 2016, pp. 547–576.
- [13] K. Ma and G. Yang, "CABAC: A content-driven attribute-based access control model for Big Data," in *Proc. Artif. Intell. Secur.*, 2019, pp. 517–529.
- [14] W. Zeng, Y. Yang, and B. Luo, "Content-based access control: Use data content to assist access control for large-scale content-centric databases," in *Proc. IEEE Int. Conf. Big Data*, Oct. 2014, pp. 701–710.
- [15] S. Kim and D. J. Wu, "Access control encryption for general policies from standard assumptions," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 471–501.
- [16] X. Wang and S. M. Chow, "Cross-domain access control encryption: Arbitrary-policy, constant-size, efficient," in *Proc. IEEE Symp. Secur. Privacy*, May 2021, pp. 388–401.
- [17] M. Sedaghat and B. Preneel, "Cross-domain attribute-based access control encryption," in *Proc. Cryptol. Netw. Secur.*, 2021, pp. 3–23.
- [18] X. Wang et al., "Attribute-based access control encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2227–2242, May/Jun. 2025.
- [19] J. Cui, B. Li, H. Zhong, G. Min, Y. Xu, and L. Liu, "A practical and efficient bidirectional access control scheme for cloud-edge data sharing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 476–488, Feb. 2022.
- [20] Q. Huang, C. Wang, and L. Chen, "Secure and fine-grained flow control for subscription-based data services in cloud-edge computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 2165–2177, May/Jun. 2023.
- [21] J. Han, L. Chen, W. Susilo, X. Huang, A. Castiglione, and K. Liang, "Fine-grained information flow control using attributes," *Inf. Sci.*, vol. 484, pp. 167–182, May 2019.
- [22] Q. Huang, L. Chen, and C. Wang, "A parallel secure flow control framework for private data sharing in mobile edge cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4638–4653, Dec. 2022.
- [23] C. Shi, Q. Huang, R. Jian, and G. Chi, "Cross-domain inner-product access control encryption for secure EMR flow in cloud edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 9866–9880, 2024.
- [24] W. Susilo, P. Jiang, J. Lai, F. Guo, G. Yang, and R. Deng, "Sanitizable access control system for secure cloud storage against malicious data publishers," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 2138–2148, May/Jun. 2022.
- [25] Y. Zhao, H. Yu, Y. Liang, M. Conti, W. Bazzi, and Y. Ren, "A sanitizable access control with policy-protection for vehicular social networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2956–2965, Mar. 2024.
- [26] P. Jiang, Q. Liu, and L. Zhu, "SanIdea: Exploiting secure blockchain-based access control via sanitizable encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1589–1600, 2024.
- [27] M. Chase and S. S. Chow, "Improving privacy and security in multi-authority attribute-based encryption," in *Proc. 16th ACM Conf. Comput. Commun. Secur.*, Nov. 2009, pp. 121–130.
- [28] Y. Xin, H. Ma, and R. Zhang, "Hodor: Robust fine-grained information flow control with full data traffic protection for cloud-edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 3074–3087, 2025.
- [29] G. Fuchsbauer, R. Gay, L. Kowalczyk, and C. Orlandi, "Access control encryption for equality, comparison, and more," in *Proc. IACR Int. Workshop Public Key Cryptogr.*, 2017, pp. 88–118.
- [30] M. Abe, J. Groth, M. Ohkubo, and M. Tibouchi, "Unified, minimal and selectively randomizable structure-preserving signatures," in *Proc. Theory Cryptogr.*, 2014, pp. 688–712.
- [31] M. Abe, J. Groth, K. Haralambiev, and M. Ohkubo, "Optimal structure-preserving signatures in asymmetric bilinear groups," in *Proc. Annu. Cryptol. Conf.*, 2011, pp. 649–666.
- [32] I. Kim, S. O. Hwang, J. H. Park, and C. Park, "An efficient predicate encryption with constant pairing computations and minimum costs," *IEEE Trans. Comput.*, vol. 65, no. 10, pp. 2947–2958, Oct. 2016.
- [33] E. Crites, M. Kohlweiss, B. Preneel, M. Sedaghat, and D. Slamanig, "Threshold structure-preserving signatures," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2023, pp. 348–382.
- [34] S. Das and L. Ren, "Adaptively secure BLS threshold signatures from DDH and co-CDH," in *Proc. Annu. Int. Cryptol. Conf.*, 2024, pp. 251–284.
- [35] G. Fuchsbauer, C. Hanser, and D. Slamanig, "Structure-preserving signatures on equivalence classes and constant-size anonymous credentials," *J. Cryptol.*, vol. 32, no. 2, pp. 498–546, Apr. 2019.
- [36] J. Groth and A. Sahai, "Efficient non-interactive proof systems for bilinear groups," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2008, pp. 415–432.
- [37] E. Ghadafi, N. P. Smart, and B. Warinschi, "Groth–Sahai proofs revisited," in *Proc. Int. Workshop Public Key Cryptogr.*, 2010, pp. 177–192.
- [38] F. Hao, "Schnorr non-interactive zero-knowledge proof," Internet Eng. Task Force, Tech. Rep. RFC 8235, Sep. 2017.
- [39] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Proc. Conf. Theory Appl. Cryptographic Techn.*, 1987, pp. 186–194.
- [40] "JPBC Library," 2023. [Online]. Available: <http://gas.dia.unisa.it/projects/jpbc/>



Caiqun Shi is currently working toward the PhD degree with the School of Cyberspace Security, Beijing University of Posts and Telecommunications. Her research focuses on data security in cloud computing.



Qinlong Huang (Member, IEEE) received the PhD degree in information security from the School of Computer Science, Beijing University of Posts and Telecommunications (BUPT), China, in 2014. In 2019, he was a Visiting Scholar with Virginia Tech, USA. He is currently an associate professor with the School of Cyberspace Security, BUPT. He is also the principal investigator of several research projects funded by the National Natural Science Foundation of China, China Computer Federation. He has authored or coauthored more than 30 papers in refereed journals and conferences, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Mobile Computing*, and *IEEE Transactions on Services Computing*. His research interests include applied cryptography, data privacy, and cyber security. He was a TPC member for more than ten IEEE conferences, including IEEE GLOBECOM 2024 and IEEE TrustCom 2024.



Chao Wang is currently working toward the PhD degree with the School of Cyberspace Security, Beijing University of Posts and Telecommunications. Her research focuses on data security.